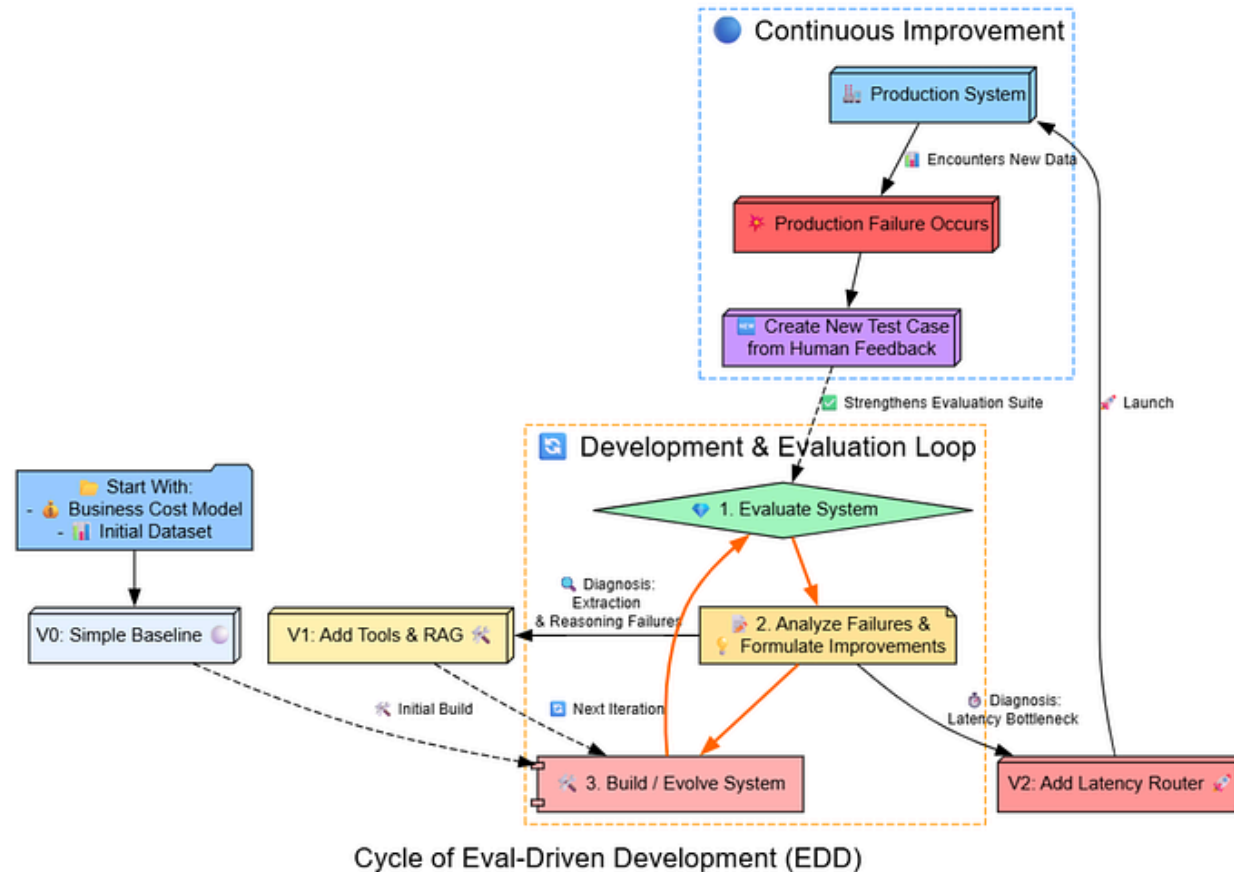


[< Go to the original](#)

Creating a Complete Evaluation Lifecycle to Build High-Confidence AI Agents

**Fareed Khan**

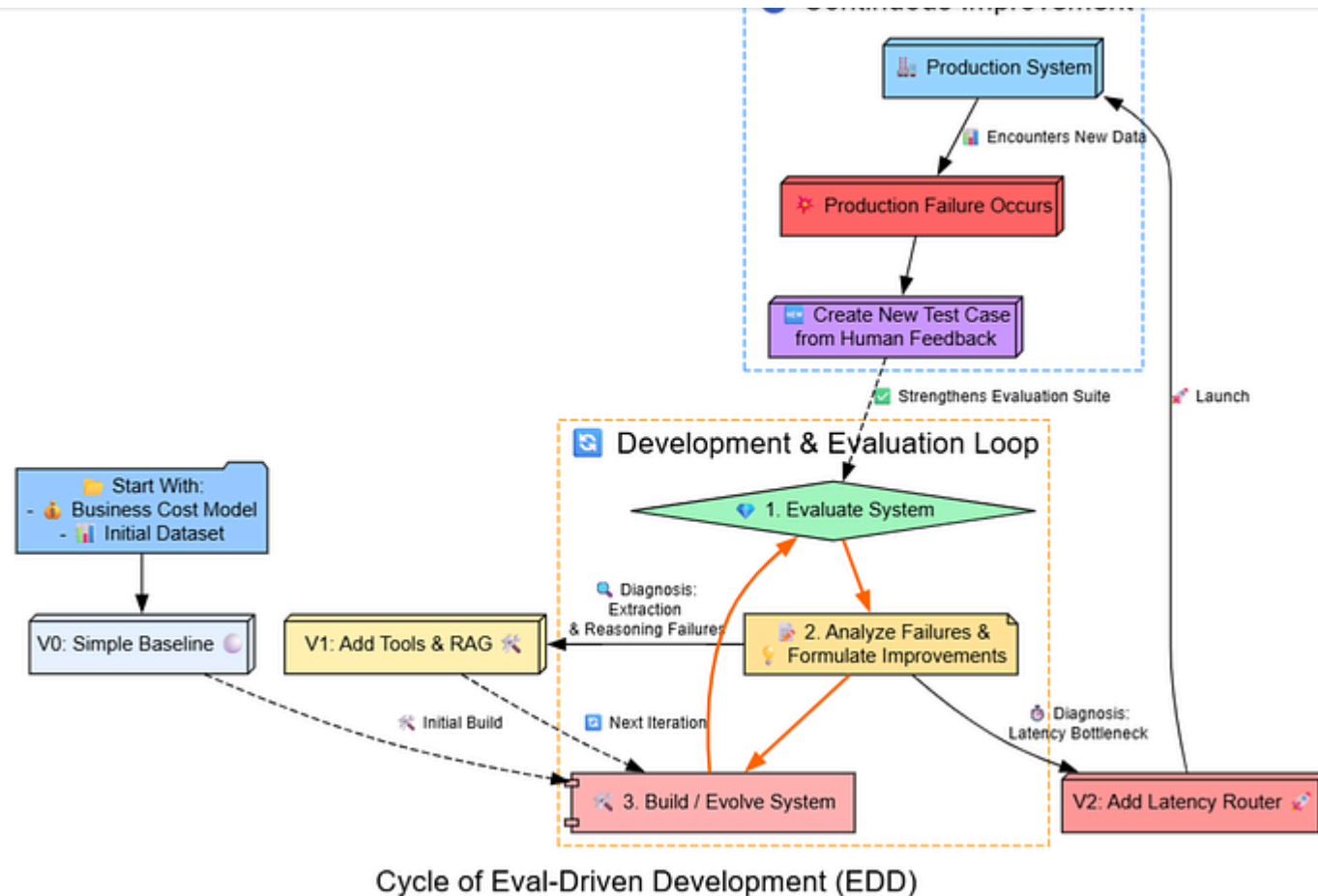
Follow

∞ Level Up Coding a11y-light ~42 min read · September 10, 2025 (Updated: September 10, 2025) ·
Free: No

Read this story for free: [link](#)

In traditional machine learning, metrics like Accuracy, Precision, and F1-score are cornerstones. They work perfectly for tasks with a finite set of correct answers. However, for generative AI and agentic systems, the output space is vast, and a simple string match against a single "correct" answer is no longer sufficient.

Without a clear way to evaluate it, improving an LLM system often means just guessing.



EDD Approach Eval Driven System (Created by [Fareed Khan](#))

This is what we call "**Trial-and-error engineering**" Engineering: tweaking a prompt, observing a few outputs, and making a subjective judgment about

Eval-Driven Development (EDD) is a better way to build. Instead of just checking if the output is correct, it uses a set of repeatable tests to measure things like quality, speed, cost, and consistency bringing more scientific steps into the process.

EDD is a productive cycle in which we ...

1. **Test:** Run your current system against a comprehensive evaluation suite.
2. **Analyze:** Dive deep into the results to identify specific failure modes.
3. **Improve:** Formulate a hypothesis and implement a targeted improvement.
4. **Repeat:** Re-run the evaluation to confirm your hypothesis and find the next bottleneck.

In this guide, I will create a complete lifecycle of building a multi-component, tool-using agent using an **Eval-Driven Development (EDD)** approach. We will start with a simple baseline and use a sophisticated evaluation suite to guide every architectural decision, evolving our system from a fragile prototype into a robust, production-ready agent.

GitHub - FareedKhan-dev/ai-eval-system: A step by step implementation of a complex AI Evaluation...

A step by step implementation of a complex AI Evaluation System Designing - FareedKhan-dev/ai-eval-system

github.com

Table of Contents

- [Setting up the Environment](#)
- [Defining Our Business Problem and Cost Model](#)
- [Our Mock Dataset and Ground Truth](#)
- [Creating the V0 Baseline](#) ◦ [Creating the V0 Extractor Agent](#) ◦ [Building the V0 Auditor Agent](#)
- [Running the V0 Pipeline on a Test Case](#)
- [Running Our Foundational Evaluation Suite](#) ◦ [Generating Predictions for the Eval Dataset](#) ◦ [Defining the V0 Grader Suite](#) ◦ [Running the Evaluation](#)
- [Analyzing the V0 failures](#)

- Augmenting Our Auditor with RAG ◦ Building the RAG Knowledge Base ◦ The V1 RAG-Augmented Auditor ◦ Running the Full V1 Pipeline
- Evaluating V1 with Slices and Operational Metrics ◦ Evaluating by Data Slices ◦ Graders for Cost, Latency, and Operational Metrics
- Synthesizing Our Findings and Planning the Next Iteration ◦ Running the V1 Advanced Evaluation ◦ Connecting to Business Cost:
- CI/CD, PromptOps, and Continuous Monitoring ◦ Step 1: Simulating Production Logging ◦ Step 2: Simulating a Production Failure ◦ Step 3: Closing the Loop with Human Feedback
- Summarizing the Cycle of Eval-Driven Development

Setting up the Environment

With this philosophy as our guide, let's set up our environment and select the tools for the job. First, let's get our dependencies installed.

Copy

```
%pip install --upgrade openai pydantic pandas scikit-learn python-dotenv rich -
```

any OpenAI-compatible API, including a local Ollama instance too.

Copy

```
import os                # For environment variables and file paths
import json              # For working with JSON data
import asyncio           # For async event loop and async/await support
from pathlib import Path  # For handling filesystem paths
from typing import List, Optional # For type hints (lists, optionals)

import pandas as pd      # For data analysis / tabular data
from openai import AsyncOpenAI # Nebius/OpenAI async client
from pydantic import BaseModel, Field # For data validation / models
from rich import print    # For rich terminal output (colors, styles)
from dotenv import load_dotenv # For loading .env environment variables

# Load environment variables from a .env file.
# Create a file named .env and add your API key like this:
# NEBIUS_API_KEY="your_api_key_here"
load_dotenv()

# Fetch the API key from environment variables
api_key = os.environ.get("NEBIUS_API_KEY")

# Configure the client to point to the Nebius AI API endpoint.
# All subsequent API calls will be routed through this custom base URL.
client = AsyncOpenAI(
```

If you are a developer, you will easily understand this code, it's just calling and initializing the LLM through an API, and you could adapt it for something like Ollama. The only difference is that I am using `AsyncOpenAI` because it helps with async tasks and non-blocking requests.

Now a key part of our strategy is using different models for different tasks. A powerful, expensive model is overkill for simple tasks, while a small model may lack the reasoning ability for complex ones. We will therefore make a principled choice for two key roles:

- **Baseline / Simple Tasks (V0 System):** `Llama-3.1-8B` This is a modern, highly capable small model. It's extremely cost-effective and fast, making it ideal for our first-pass system. It will serve as our `gpt-4o-mini` equivalent.
- **Advanced / Grading Tasks (V1 System & Evals):** `Qwen/Qwen3-235B-A22B-Instruct-2507`, This is one of the highest-performing open-source models available. Its large size and explicit support for reasoning, math, and structured JSON output make it a perfect candidate for our advanced V1 agent and, crucially, for our model-based graders. A powerful, impartial judge is essential for high-quality evaluations. It will serve as our `gpt-4o` equivalent.

Defining Our Business Problem and Cost Model

With our philosophy and environment in place, we can now move onto the first step in any successful AI project is to deeply understand and quantify the business problem we are trying to solve.

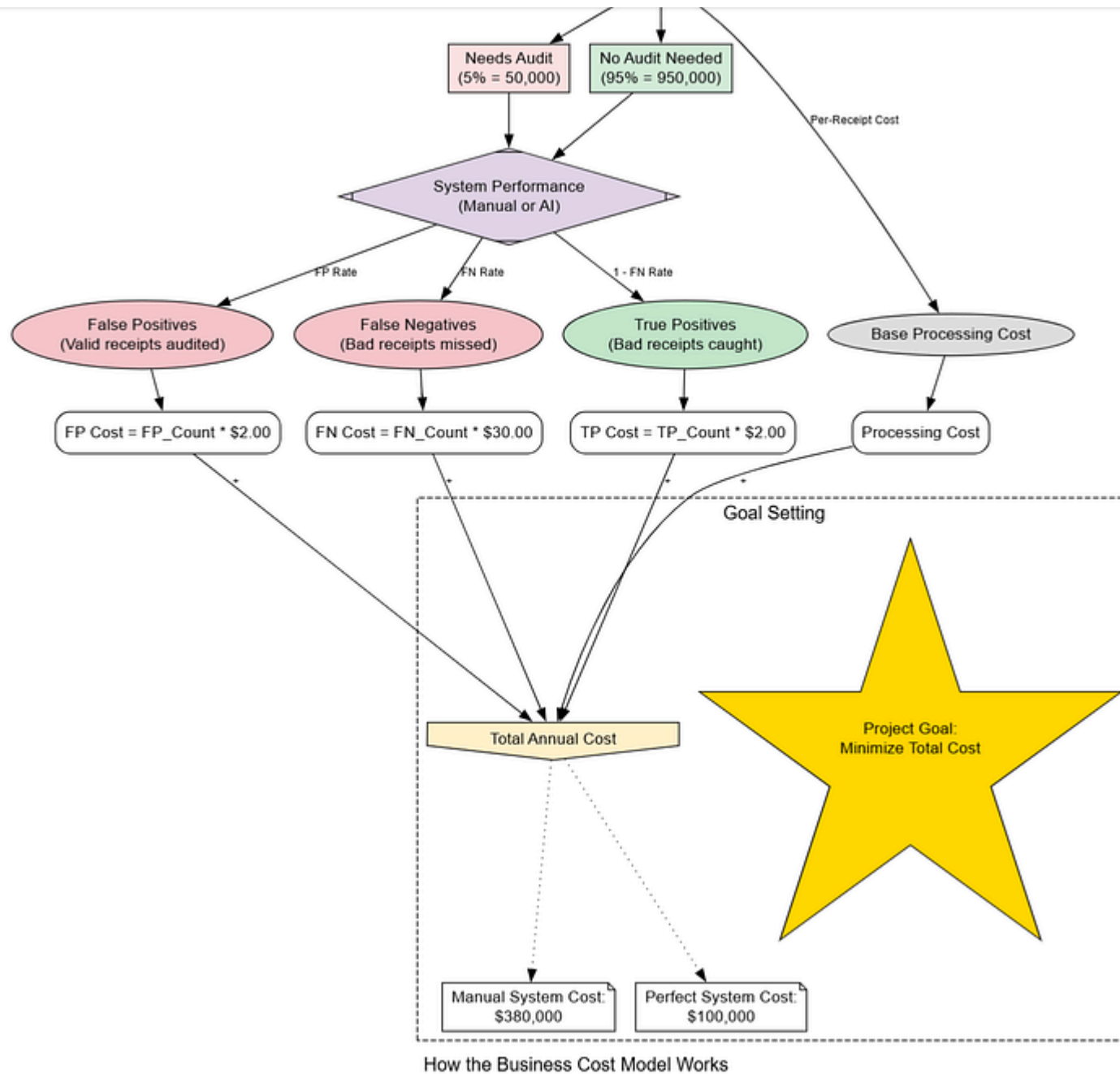
Now let's understand our scenario: A company processes thousands of employee expense receipts monthly. The manual review process is slow, expensive, and prone to human error. Our mission is to build an AI agent that automates this workflow, intelligently flagging only the necessary receipts for a final human audit.

We do know that real businesses need more than accuracy a production-ready system must be evaluated across multiple factors ...

- **Correctness:** We must minimize both missed non-compliant receipts (False Negatives) and incorrectly flagged valid receipts (False Positives).
- **Cost-Effectiveness:** The total cost of the AI system, including its operational expenses and any errors it makes, must be significantly lower than the

Freedium[ko-fi](#) [librepay](#) [patreon](#)

-
- **Speed.** The system must be fast enough to avoid delaying employee reimbursements.



To make rational, data-driven decisions about our agent's architecture, we must first build a business cost model.

Before coding, we measure the business impact. The model turns performance (false positives and false negatives) into dollar values, giving us a clear direction.

Our Assumptions:

- Receipts per year: 1,000,000
- Cost of human audit per receipt: \$2.00
- Average loss from a missed non-compliant receipt (a False Negative): \$30.00
- Percentage of receipts that genuinely require an audit: 5%
- Cost of the current manual process (per receipt, before audit costs): \$0.20
- Cost of our AI system (per receipt): We will estimate this later based on token usage.

Let's translate these assumptions into a Python function that can calculate the total annual cost of any given system, whether it's our current manual one or a future AI agent.

```
def calculate_business_cost(
    fp_rate: float,
    fn_rate: float,
    per_receipt_processing_cost: float,
    receipt_count: int = 1_000_000,
    audit_fraction: float = 0.05,
    human_audit_cost: float = 2.0,
    missed_audit_loss: float = 30.0
) -> float:
    """Models the total annual cost of an expense auditing system."""

    needs_audit_count = receipt_count * audit_fraction
    no_audit_needed_count = receipt_count * (1 - audit_fraction)

    # Cost of False Positives: valid receipts sent to human audit
    false_positives = no_audit_needed_count * fp_rate
    fp_cost = false_positives * human_audit_cost

    # Cost of False Negatives: bad receipts that were missed
    false_negatives = needs_audit_count * fn_rate
    fn_cost = false_negatives * missed_audit_loss

    # Cost of True Positives: correctly identified audits
    true_positives = needs_audit_count * (1 - fn_rate)
    tp_cost = true_positives * human_audit_cost

    # Total processing cost for the AI system or manual labor
    processing_cost = receipt_count * per_receipt_processing_cost
```

```
# Baseline: Our current human-driven process (let's assume 2% FP, 3% FN)
current_system_cost = calculate_business_cost(fp_rate=0.02, fn_rate=0.03, per_receipt_cost=100)
perfect_system_cost = calculate_business_cost(fp_rate=0, fn_rate=0, per_receipt_cost=100)

print(f"Cost of current manual system: ${current_system_cost:,.2f}")
print(f"Cost of a theoretically perfect system: ${perfect_system_cost:,.2f}")
```

This `calculate_business_cost` function is the core of our business case. It breaks down the total cost into its key components: the cost of incorrect decisions (False Positives and False Negatives), the cost of correct decisions (auditing True Positives), and the baseline processing cost.

By running this model, we can establish our two most important benchmarks:

1. the cost of our current process ...
2. and the theoretical minimum cost.

Copy

```
##### OUTPUT OF ABOVE CODE #####
Cost of current manual system: $380,000.00
Cost of a theoretically perfect system: $100,000.00
```

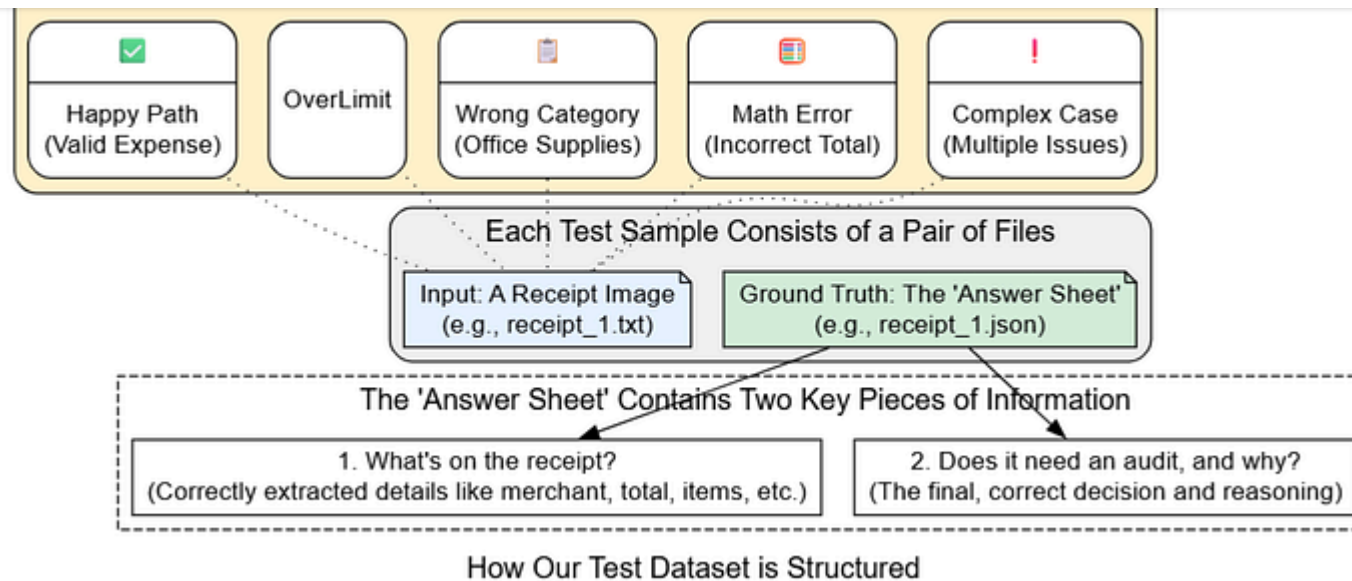
A theoretically perfect system one that makes no mistakes and has zero processing cost, would still cost **\$100,000 per year**, because 5% of receipts genuinely require an audit and must still be paid for.

Our goal is now defined to build an AI system that gets as close to this \$100,000 baseline as possible, while keeping its own processing costs low. This financial framework will allow us to objectively measure the value of every improvement we make.

With our mission scoped and quantified, we now need data to train and test our agent. Let's prepare that.

Our Mock Dataset and Ground Truth

For this project, to ensure our workflow is fully self-contained and reproducible, we will programmatically create a small, high-quality mock dataset.



Our Real World Mock Data (Created by [Fareed Khan](#))

In a real-world scenario, this step would involve collecting thousands of real receipt images and having domain experts, like accountants, meticulously label them to create our ground truth.

Our dataset will be designed to test our agent against a variety of common challenges. First, we need to define the data structures for both the information we'll extract from the receipts and the final audit decision.

type-safe structure that we can use to constrain the LLM's output, a technique for building reliable systems.

Let's start by defining the smallest, most fundamental pieces of our data structure: the location and the individual line items on a receipt.

Copy

```
class Location(BaseModel):
    # Optional fields allow the model to return None if the info is not on the
    city: Optional[str] = None
    state: Optional[str] = None

class LineItem(BaseModel):
    description: str
    quantity: float
    price: float # Price per item
    total: float
```

Here, we have created two simple classes. `Location` is straightforward, but notice we have made its fields `Optional`. This is a practical consideration, as not all receipts will have a city or state, and this tells our system (and the LLM) that it's

Now, we can compose these building blocks into a larger structure that represents a full receipt.

Copy

```
class ReceiptDetails(BaseModel):  
    merchant: str  
    location: Location  
    items: List[LineItem]  
    subtotal: Optional[float] = None  
    tax: float  
    total: float  
    handwritten_notes: List[str] = []
```

Our `ReceiptDetails` model brings everything together. It includes the merchant's name, a nested `Location` object, and a `List` of `LineItem` objects. This hierarchical structure is a clean way to represent the complex data on a real receipt.

Finally, we define the schema for the most critical output of our agent: the final audit decision.

```
class AuditDecision(BaseModel):  
    # We use Field(description=...) to provide extra context to the LLM about v  
    # This acts as a form of prompt engineering directly within the data schema  
    not_travel_related: bool = Field(description="True if the receipt is not for travel  
    amount_over_limit: bool = Field(description="True if the total amount exceeds the limit  
    math_error: bool = Field(description="True if line items + tax do not sum to total  
    handwritten_x: bool = Field(description="True if there is an 'X' in handwritten  
    reasoning: str  
    needs_audit: bool
```

This `AuditDecision` class is where we embed our business logic directly into the schema. The `Field(description=...)` is a powerful technique. This description isn't just a comment for us developers, it's passed to the LLM as part of the tool's schema, giving it explicit instructions on what each boolean flag represents. This significantly improves the reliability of the final structured output.

With our data schemas defined, we can now create a function to generate and save our mock data. This function will create text files to simulate our receipt "images" and corresponding JSON files for our ground truth labels.

Copy

```
# --- Function to Create Mock Data ---  
def setup_mock_dataset():  
    """Generates a small, local dataset for demonstration purposes."""
```

```
gt_dir = data_dir / ground_truth

img_dir.mkdir(parents=True, exist_ok=True)
gt_dir.mkdir(parents=True, exist_ok=True)

# This dictionary defines our test cases, covering various scenarios.
mock_data = {
    "receipt_1": { # Simple, valid travel expense
        "image_content": "Shell Gas Station\n123 Fuel Rd, Houston, TX\nUnleaded",
        "ground_truth": {
            "details": ReceiptDetails(merchant="Shell", location=Location(city="Houston", state="TX", address="123 Fuel Rd")),
            "audit": AuditDecision(not_travel_related=False, amount_over_limit=False)
        }
    },
    "receipt_2": { # Over limit
        "image_content": "Luxury Hotel\n456 Park Ave, New York, NY\nRoom Service",
        "ground_truth": {
            "details": ReceiptDetails(merchant="Luxury Hotel", location=Location(city="New York", state="NY", address="456 Park Ave")),
            "audit": AuditDecision(not_travel_related=False, amount_over_limit=True)
        }
    },
    "receipt_3": { # Not travel-related
        "image_content": "Office Supplies Inc.\n789 Desk Blvd, Chicago, IL\nPrinter Paper",
        "ground_truth": {
            "details": ReceiptDetails(merchant="Office Supplies Inc.", location=Location(city="Chicago", state="IL", address="789 Desk Blvd")),
            "audit": AuditDecision(not_travel_related=True, amount_over_limit=False)
        }
    },
    "receipt_4": { # Math error
```

```

        "details": ReceiptDetails(merchant="Corner Cafe", location="Loc
        "audit": AuditDecision(not_travel_related=False, amount_over_li
    },
    },
    "receipt_5": { # Handwritten note and not travel-related
        "image_content": "Hardware Store\n202 Tool St, Austin, TX\nHammer:
        "ground_truth": {
            "details": ReceiptDetails(merchant="Hardware Store", location=l
            "audit": AuditDecision(not_travel_related=True, amount_over_li
        }
    }
}

print(f"Mock dataset created at: {data_dir}")
for name, data in mock_data.items():
    # In this notebook, we simulate images by using plain text files.
    img_path = img_dir / f"{name}.txt"
    gt_path = gt_dir / f"{name}.json"

    img_path.write_text(data["image_content"])
    gt_path.write_text(json.dumps(data["ground_truth"], indent=2))

    print(f"  - {img_path}")
    print(f"  - {gt_path}")

return data_dir

# Create the dataset on disk for our notebook to use.
DATA_DIR = setup_mock_dataset()

```

The `setup_mock_dataset` function is the engine of our data preparation. The `mock_data` dictionary is where we have carefully engineered our test cases. Each key (`receipt_1` , `receipt_2` , etc.) represents a unique scenario designed to test a specific audit rule:

- `receipt_1` is our "happy path" a clean, compliant expense.
- `receipt_2` tests the "amount over limit" rule.
- `receipt_3` tests the "not travel-related" rule.
- `receipt_4` tests for a clear mathematical error.
- `receipt_5` is our complex edge case, combining multiple failure points (non-compliant and a handwritten void).

This variety is crucial for building a meaningful evaluation. A system that only works on the happy path is not production-ready.

Copy

```
##### OUTPUT OF ABOVE CODE #####  
Mock dataset created at: temp_data  
- temp_data/images/receipt_1.txt  
- temp_data/ground_truth/receipt_1.json
```

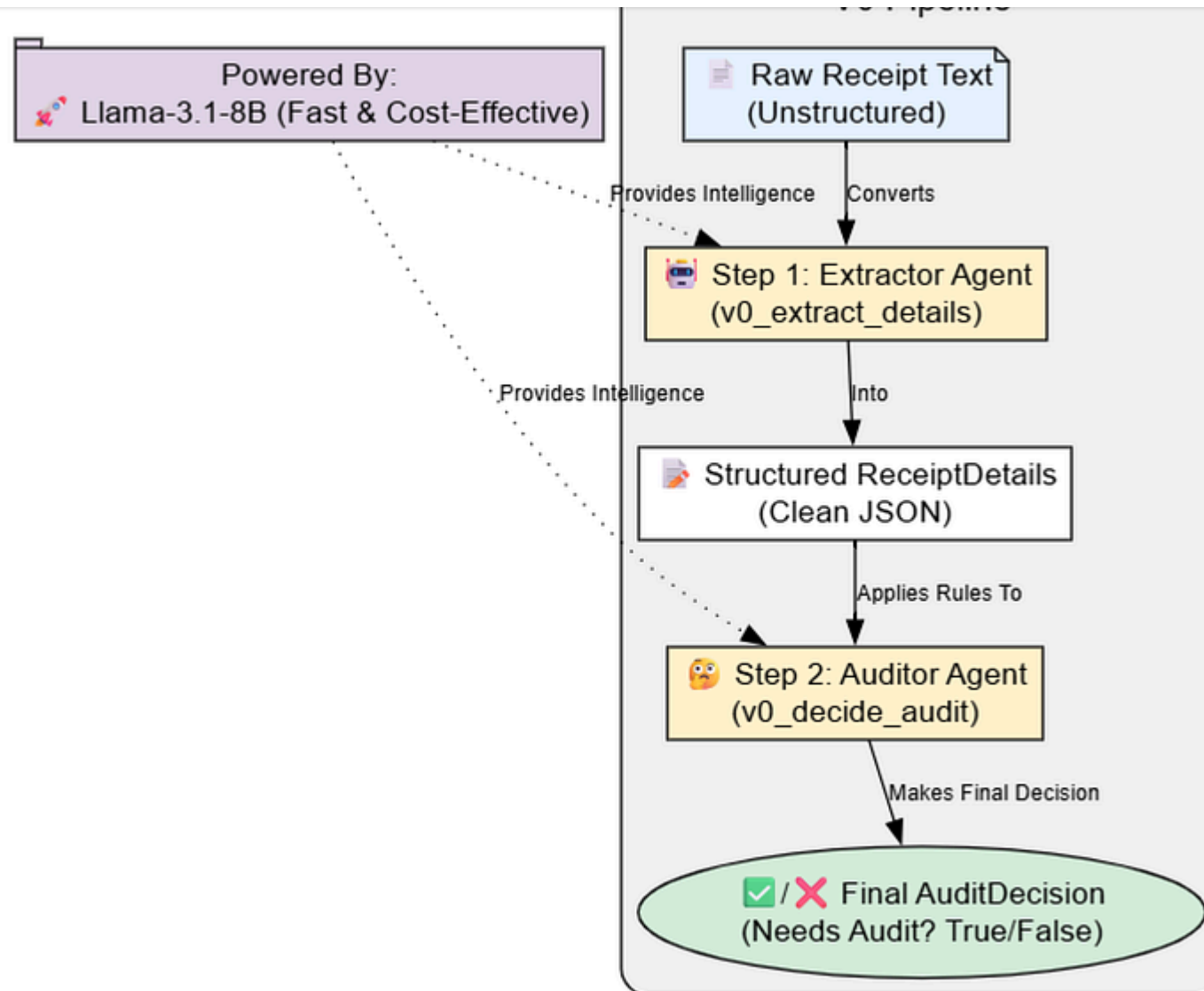
```
- temp_data/images/receipt_3.txt
- temp_data/ground_truth/receipt_3.json
- temp_data/images/receipt_4.txt
- temp_data/ground_truth/receipt_4.json
- temp_data/images/receipt_5.txt
- temp_data/ground_truth/receipt_5.json
```

The output confirms that our test data has been successfully created and saved to disk. We now have our "images" (the `.txt` files containing the raw receipt text) and our expert-labeled answers (the `.json` files containing the ground truth `ReceiptDetails` and `AuditDecision`).

Now that our business problem quantified and our dataset is prepared, we can now going to build and evaluate our first system which we can call **V0 baseline**.

Creating the V0 Baseline

With our problem defined and our data ready, it's time to build our first system. In Eval-Driven Development, the initial version, or V0, follows a core principle:



The V0 Baseline System: A Simple, Two-Step Chain

V0 Baseline (Created by [Fareed Khan](#))

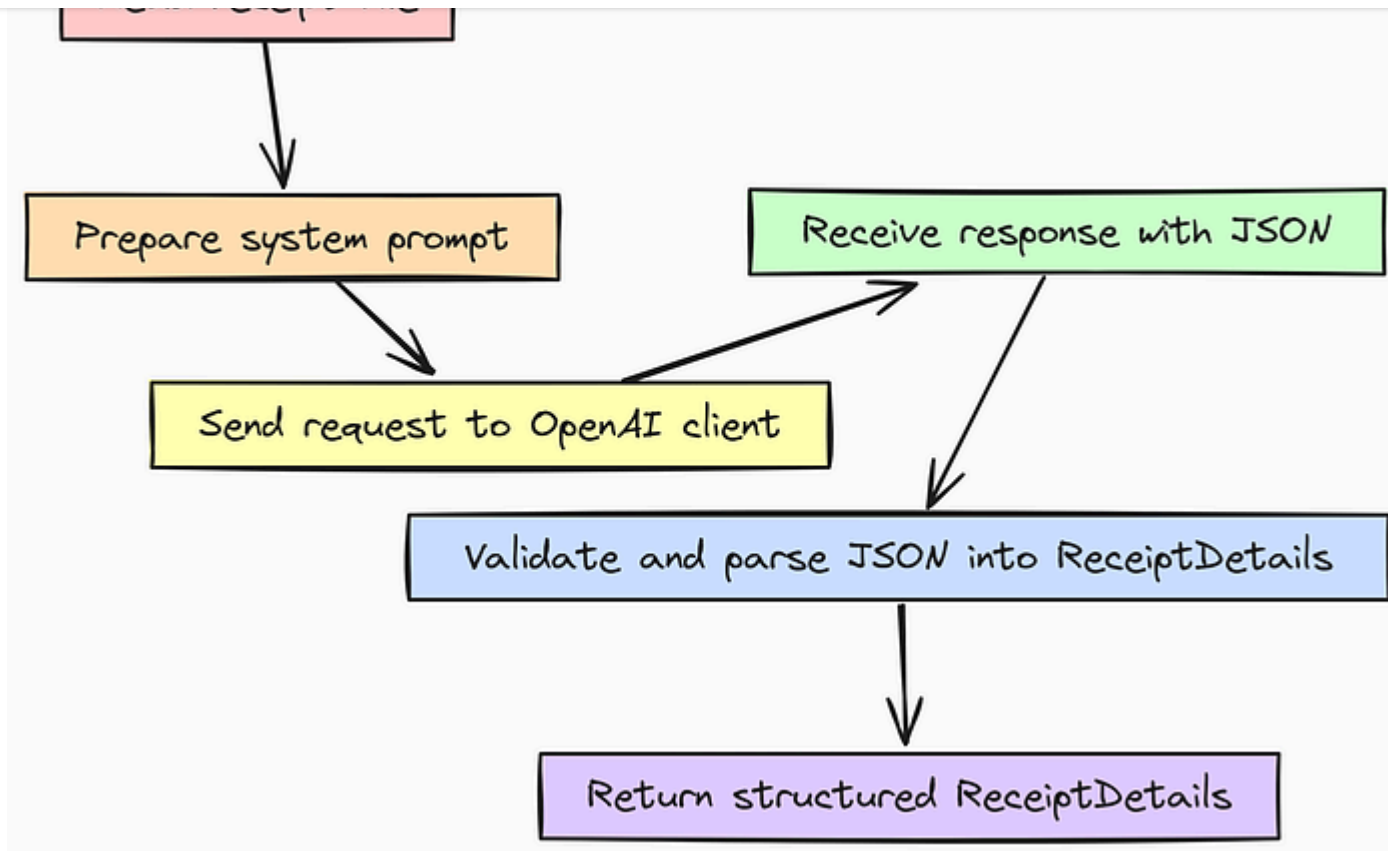
2. This baseline allows us to run our initial evaluations and uncover the most significant weaknesses, which will then guide our future, more targeted development efforts.

We will create a simple, linear chain, the raw receipt text will be fed into an **Extractor** agent, its structured output will be passed to an **Auditor** agent, and the Auditor will produce the final decision. For this V0 system, we will use our fast and cost-effective `meta/Meta-Llama-3.1-8B-Instruct` model.

Let's translate this simple chain into code. Our V0 system will have two main components: an agent for extracting details and an agent for making the audit decision.

Creating the V0 Extractor Agent

First, we will create the `v0_extract_details` function. This agent's only job is to take the raw text from our simulated receipt image and convert it into the structured `ReceiptDetails` Pydantic model we defined earlier.

V0 Agent Process (Created by [Fareed Khan](#))

Copy

```
async def v0_extract_details(image_path: Path, model: str = "meta/Meta-Llama-3.1-70B-Instruct", prompt: str = "V0 Extractor: A simple prompt to get structured data from text."):
```

Read the content from our simulated image file.

```
image_content = image_path.read_text()
```

```
    """You are an expert receipt processing assistant.
    "Extract the key details from the provided receipt text and format it as a JSON object.
    """

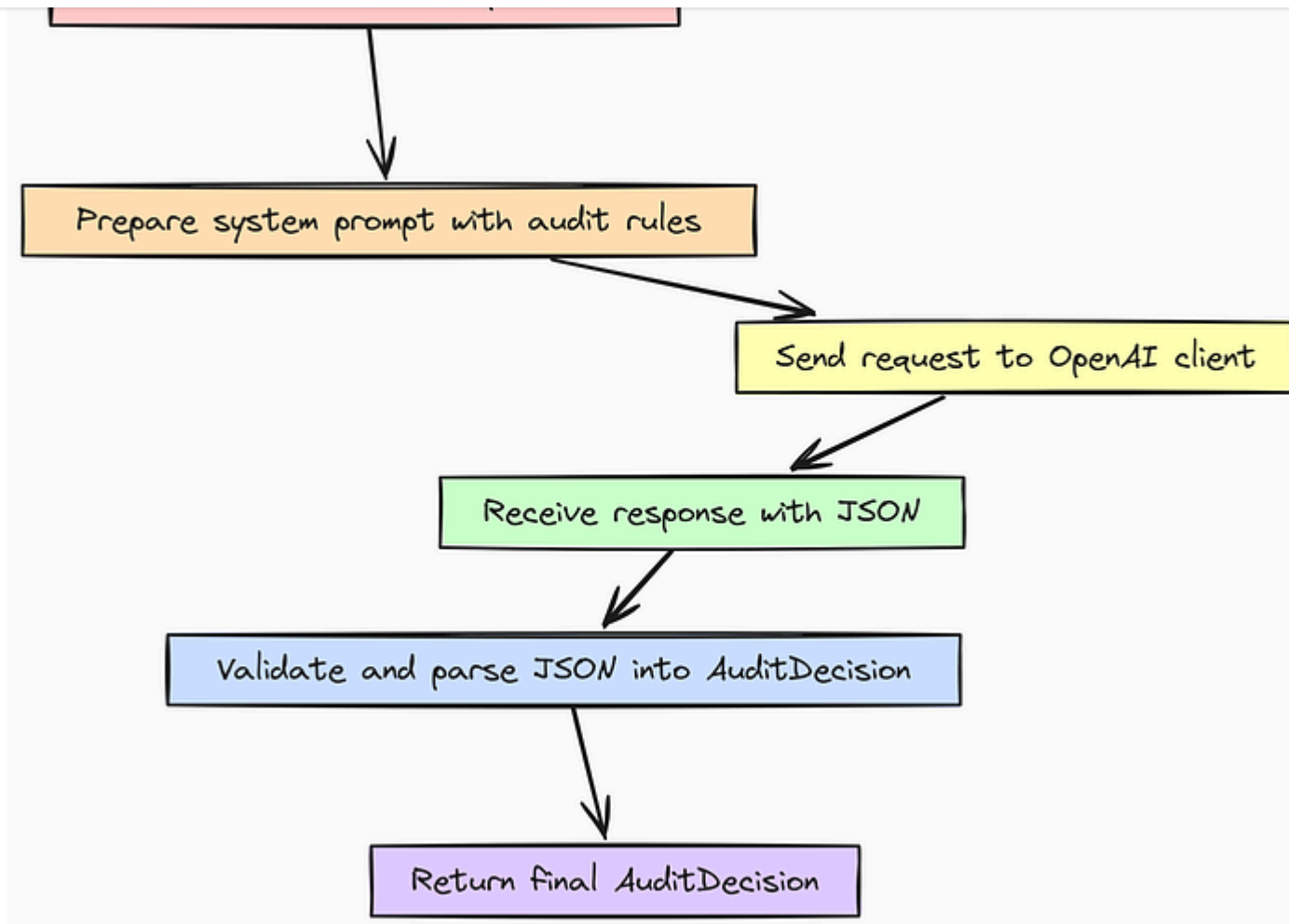
    # The 'openai' library can be used with any OpenAI-compatible API.
    # By using `response_model`, we instruct the client to handle the JSON parsing.
    # This is a form of 'output parsing' or 'constrained decoding'.
    response = await client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": f"Here is the receipt content:\n\n{image_content}"},
        ],
        response_model=ReceiptDetails # This ensures the output is a valid ReceiptDetails object
    )
    return response
```

The logic here is intentionally straightforward. We provide a simple `system_prompt` that clearly states the agent's role. The most important part of this function is the `response_model=ReceiptDetails` argument. By passing our Pydantic class directly to the client, we offload the complex and error-prone task of parsing potentially malformed JSON.

The client library handles the validation, which is making sure that the function will either return a valid `ReceiptDetails` object or raise an error, making our

Building the V0 Auditor Agent

Next, we create the `v0_decide_audit` function. This agent receives the structured `ReceiptDetails` object from the extractor and is responsible for applying our business rules to make the final `AuditDecision`.



Auditor Agent (Created by [Fareed Khan](#))

Copy

```
async def v0_decide_audit(receipt_details: ReceiptDetails, model: str = "meta/M\n    """V0 Auditor: A simple prompt to make a decision based on structured data.
```

```

    You are an expert expense auditor. Based on the provided receipt details,
    "determine if an audit is required based on these rules:\n"
    "1. NOT_TRAVEL_RELATED: Is the expense for something other than travel?\n"
    "2. AMOUNT_OVER_LIMIT: Is the total over $75?\n"
    "3. MATH_ERROR: Do the line item totals + tax not add up to the final total?\n"
    "4. HANDWRITTEN_X: Is there a handwritten 'X' indicating a voided item?\n"
    "If ANY of these are true, the receipt needs an audit. Provide your reasoning."

    )

    response = await client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            # We serialize the Pydantic object back into a JSON string for the model
            {"role": "user", "content": receipt_details.model_dump_json(indent=2)},
        ],
        response_model=AuditDecision # Ensure the output conforms to our AuditDecision schema
    )
    return response

```

In this function, we explicitly provide all the business rules directly in the `system_prompt`. This is the simplest way to inject logic into an LLM. We take the `ReceiptDetails` object, serialize it back into a clean JSON string using `.model_dump_json()`, and pass it to the model. Again, we use `response_model=AuditDecision` to guarantee a valid, structured output.

Running the V0 Pipeline on a Test Case

Now, let's wire these two agents together and run them on a single, interesting example from our dataset: `receipt_4.txt`, the one with a known math error. This will give us a quick gut check to see if the system can handle a non-trivial case.

Copy

```
async def run_v0_pipeline(image_path: Path):
    """Executes the full V0 pipeline: extraction then auditing."""
    extracted_details = await v0_extract_details(image_path)
    audit_decision = await v0_decide_audit(extracted_details)
    return extracted_details, audit_decision

# Run V0 on a single example to see it in action.
# We choose the receipt with a known math error to see if the simple system can
example_path = IMAGE_DIR / "receipt_4.txt"
print(f"Running V0 pipeline on {example_path.name} (the one with the math error)")

# Await the asynchronous pipeline.
v0_example_details, v0_example_audit = await run_v0_pipeline(example_path)

print("\nV0 Extracted Details:")
print(v0_example_details)
print("\nV0 Audit Decision:")
print(v0_example_audit)
```

Copy

```
##### OUTPUT #####
```

```
Running V0 pipeline on receipt_4.txt (the one with the math error)...
```

```
V0 Extracted Details:
```

```
ReceiptDetails(
```

```
    merchant='Corner Cafe',
```

```
    location=Location(city='San Francisco', state='CA'),
```

```
    items=[
```

```
        LineItem(description='Sandwich', quantity=1.0, price=10.0, total=10.0),
```

```
        LineItem(description='Coffee', quantity=1.0, price=4.0, total=4.0),
```

```
    ],
```

```
    subtotal=None,
```

```
    tax=1.0,
```

```
    total=16.0,
```

```
    handwritten_notes=[],
```

```
)
```

```
V0 Audit Decision:
```

```
AuditDecision(
```

```
    not_travel_related=False,
```

```
    amount_over_limit=False,
```

```
    math_error=True,
```

```
    handwritten_x=False,
```

```
    reasoning='The sum of the items ($10.00 + $4.00) plus tax ($1.00) is $15.00'
```


The V0 system seems to work on this single example! The Llama 8B model correctly extracted all the data from the text and, more impressively, the auditor agent correctly identified the math error and flagged the receipt for an audit.

However, we cannot trust a single data point. This success might just be a lucky coincidence. To truly understand our system's performance, we must build a formal evaluation suite and test it against our entire dataset to systematically identify where it breaks. It's time to put our V0 system through the first gauntlet.

Running Our Foundational Evaluation Suite

Our V0 system passed a single spot-check, but this gives us a false sense of security. To truly understand its capabilities and limitations, we must move from anecdotal evidence to systematic testing. This is where we build our foundational evaluation suite.

A robust evaluation is composed of several key elements:

1. an **Input** (our receipt text)
2. **Prediction** (the agent's output)

The core of the evaluation lies in **Graders** functions that compare the prediction to the reference and produce a score.

Our graders will fall into two categories:

- **Deterministic:** Simple, rule-based logic like `prediction.total == reference.total`. These are fast, cheap, and perfect for checking objective facts.
- **Model-Based:** Using a powerful LLM (our `Qwen-235B` model) as an impartial judge to score complex, nuanced aspects, like the quality of the generated reasoning.

First, we need to run our **V0 pipeline** on all our test samples to collect the inputs, predictions, and ground truth references into a single, unified structure for evaluation.

Generating Predictions for the Eval Dataset

We'll start by creating an `EvaluationRecord` class to hold all the data for a single test case. This keeps our evaluation logic clean and organized.

Copy

```
sample_id: str
input_content: str
predicted_details: ReceiptDetails
predicted_audit: AuditDecision
reference_details: ReceiptDetails
reference_audit: AuditDecision
```

Now, we will create an asynchronous function, `generate_predictions_for_eval`, to iterate through our entire dataset, run the V0 pipeline on each sample, and package the results into a list of these `EvaluationRecord` objects.

Copy

```
async def generate_predictions_for_eval(image_dir: Path, ground_truth_dir: Path):
    """Runs the V0 pipeline over the entire dataset to prepare for evaluation."""
    records = []
    # Get all sample files, sorted to ensure consistent order.
    image_paths = sorted(list(image_dir.glob("*.txt")))
    print(f"Generating predictions for {len(image_paths)} samples...")

    # We can run these in parallel, but for clarity in the notebook, we'll run
    for image_path in image_paths:
        sample_id = image_path.stem
        gt_path = ground_truth_dir / f"{sample_id}.json"

        # Load ground truth from the JSON file.
```

```
reference_details = ReceiptDetails(**ground_truth_data["details"])
reference_audit = AuditDecision(**ground_truth_data["audit"])

# Get predictions by running the V0 pipeline.
predicted_details, predicted_audit = await run_v0_pipeline(image_path)

records.append(EvaluationRecord(
    sample_id=sample_id,
    input_content=image_path.read_text(),
    predicted_details=predicted_details,
    predicted_audit=predicted_audit,
    reference_details=reference_details,
    reference_audit=reference_audit
))
print(f"Processed {image_path.name}")

print("Finished generating predictions.")
return records

# This step involves API calls and may take a moment.
# We will store the results in a variable for our evaluation run.
eval_dataset = await generate_predictions_for_eval(IMAGE_DIR, GROUND_TRUTH_DIR)
```

This function is the workhorse of our evaluation setup. It systematically processes each receipt, pairing the V0 agent's `predicted_details` and

Copy

```
##### OUTPUT #####  
Generating predictions for 5 samples...  
Processed receipt_1.txt  
Processed receipt_2.txt  
Processed receipt_3.txt  
Processed receipt_4.txt  
Processed receipt_5.txt  
Finished generating predictions.
```

Now that with our `eval_dataset` now populated, we can now full able to create graders and run the full evaluation.

Defining the V0 Grader Suite

In a real project, this might be a configuration file for a platform like **OpenAI Evals**. Here, we will define them as Python functions to simulate the run locally, which clearly demonstrates the core logic.

We'll start with simple, deterministic graders for objective facts.

Copy

```
def grade_total_amount(record: EvaluationRecord) -> float:
    """Checks if the predicted total amount matches the reference."""
    # For financial data, an exact match is expected.
    return 1.0 if record.predicted_details.total == record.reference_details.total else 0.0

def grade_merchant_name(record: EvaluationRecord) -> float:
    """Checks for a case-insensitive, whitespace-stripped match of the merchant name."""
    # Normalizing strings makes the check robust to minor LLM formatting differences.
    pred = record.predicted_details.merchant.lower().strip()
    ref = record.reference_details.merchant.lower().strip()
    return 1.0 if pred == ref else 0.0

def grade_final_audit_decision(record: EvaluationRecord) -> float:
    """Checks if the most important final output, needs_audit, is correct."""
    return 1.0 if record.predicted_audit.needs_audit == record.reference_audit.needs_audit else 0.0

def grade_audit_reason_amount(record: EvaluationRecord) -> float:
    """Checks if the 'amount_over_limit' flag was correctly identified."""
    return 1.0 if record.predicted_audit.amount_over_limit == record.reference_audit.amount_over_limit else 0.0

def grade_audit_reason_math_error(record: EvaluationRecord) -> float:
    """Checks if the 'math_error' flag was correctly identified."""
    return 1.0 if record.predicted_audit.math_error == record.reference_audit.math_error else 0.0
```

Now if you look each of the function, each one targets a specific, measurable aspect of the agent performance, from the accuracy of extracted fields like `total` and `merchant` to the correctness of the final audit flags.

Copy

```
# --- Model-Based Grader Function ---

async def grade_reasoning_quality(record: EvaluationRecord, model: str = "Qwen",
    """Use a powerful LLM to judge the quality of the reasoning string."""
    prompt = f"""
    You are an evaluation assistant. Compare the predicted reasoning for an audit decision.
    Score 1 if the predicted reasoning logically and correctly justifies the audit decision.
    Score 0 if the reasoning is flawed, irrelevant, or misses the key point.

    GROUND TRUTH REASONING:
    {record.reference_audit.reasoning}

    PREDICTED REASONING:
    {record.predicted_audit.reasoning}

    Does the predicted reasoning correctly justify the decision? Respond with 0 or 1.
    """
    response = await client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        max_tokens=1, # We only need a single token '1' or '0'
        temperature=0 # Use 0 temperature for deterministic grading
    )
    score_str = response.choices[0].message.content
    try:
```

```
return 0.0 # Fail the grade if the output is not a valid number.
```

This is our first **LLM-as-a-Judge**. It's critical to use a highly capable model for this task (Qwen-235B in our case) to ensure the evaluation itself is accurate. We use a low temperature and constrained output (max_tokens=1) to make the grading as deterministic as possible.

Running the Evaluation

Finally, we will create an orchestrator function to run our entire suite of graders across the dataset and present the results in a clean DataFrame.

Copy

```
# --- Evaluation Runner ---
async def run_simulated_eval(dataset: List[EvaluationRecord]):
    """Orchestrates the running of all graders against the dataset."""
    grader_suite = {
        "Extraction/Total Amount": grade_total_amount,
        "Extraction/Merchant Name": grade_merchant_name,
        "Audit/Final Decision Correct": grade_final_audit_decision,
        "Audit/Reason: Over Limit": grade_audit_reason_amount,
        "Audit/Reason: Math Error": grade_audit_reason_math_error,
        "Audit/Reasoning Quality (Model-Graded)": grade_reasoning_quality,
    }
```



```
    for record in dataset:
        sample_scores = {"sample_id": record.sample_id}
        for name, func in grader_suite.items():
            if asyncio.iscoroutinefunction(func):
                score = await func(record)
            else:
                score = func(record)
            sample_scores[name] = score
        results.append(sample_scores)

    return pd.DataFrame(results).set_index("sample_id")

# Run the evaluation and get the detailed per-sample results.
v0_results_df = await run_simulated_eval(eval_dataset)

# Calculate the summary statistics (pass rate) for each grader.
v0_summary = pd.DataFrame(v0_results_df.mean(), columns=["Pass Rate"])

display(v0_results_df)
display(v0_summary)
```

This final block executes the evaluation. Let's look at the results.

	Amount	Name	Correct	Over Limit	
receipt_1	1.0	1.0	1.0	1.0	
receipt_2	1.0	1.0	1.0	1.0	
receipt_3	1.0	0.0	1.0	1.0	
receipt_4	1.0	1.0	1.0	1.0	
receipt_5	0.0	1.0	0.0	1.0	

	Pass Rate
Extraction/Total Amount	0.8
Extraction/Merchant Name	0.8
Audit/Final Decision Correct	0.8
Audit/Reason: Over Limit	1.0
Audit/Reason: Math Error	1.0
Audit/Reasoning Quality (Model-Graded)	0.8

1.md hosted with ❤️ by GitHub

[view raw](#)

The evaluation results provide a clear, quantitative verdict on our V0 system's performance. The detailed, per-sample DataFrame immediately flags receipt_3

- On receipt_3 , the system achieved a score of 0.0 for Extraction/Merchant Name , indicating a specific failure in parsing the merchant "Office Supplies Inc."
- On receipt_5 , the failures are systemic. A 0.0 on Extraction/Total Amount shows the initial data parsing was incorrect. This error cascaded, leading to a 0.0 on Audit/Final Decision Correct and a 0.0 on Audit/Reasoning Quality (Model-Graded) .

With these results in hand, we can now move to the most important part of the EDD cycle, analyzing the failures.

Analyzing the VO failures

While performing analysis we basically replaces subjective feelings with hard data, forcing us to confront the real weaknesses in our system.

Our summary table shows pass rates of 80% across several key metrics. While not a complete disaster, an 80% success rate is unacceptable for a production system that handles financial data.

problems stem from `receipt_5`, which triggered a cascade of failures.

A high-level summary isn't enough. We need to perform a root cause analysis.

To move from what failed to why it failed, we must go deep into the specific `EvaluationRecord` for our problem case, `receipt_5`, and compare its predicted outputs directly against our ground truth.

Let's write a small piece of code to isolate this specific record and print a clear diagnostic report.

Copy

```
# Isolate the specific record that failed multiple tests.
failure_record = next(r for r in eval_dataset if r.sample_id == "receipt_5")

print("Root Cause Analysis for receipt_5")
print("-----")
print(f"Predicted Total: {failure_record.predicted_details.total}")
print(f"Reference Total: {failure_record.reference_details.total}")
print(f"\nPredicted Final Audit: {failure_record.predicted_audit.needs_audit}")
print(f"Reference Final Audit: {failure_record.reference_audit.needs_audit}")
print("\nPredicted Reasoning:")
print(failure_record.predicted_audit.reasoning)
```

This code snippet acts as our diagnostic tool. We iterate through our `eval_dataset` to find the record where `sample_id` is "receipt_5". We then print a side-by-side comparison of the most critical predicted fields versus their reference (ground truth) values. This allows for a direct, unambiguous analysis of the agent's errors.

Copy

```
##### OUTPUT #####
```

```
Root Cause Analysis for receipt_5
```

```
-----
```

```
Predicted Total: 20.0
```

```
Reference Total: 21.6
```

```
Predicted Final Audit: False
```

```
Reference Final Audit: True
```

```
Predicted Reasoning:
```

```
The receipt is for a hardware store, which is not a travel-related expense. The
```

```
Reference Reasoning:
```

```
Not travel related and contains a voiding 'X' note.
```

- 1. A Severe Extraction Failure:** The first two lines reveal the initial error. The agent predicted a total of \$20.00 when the correct total was \$21.60. Looking back at the raw receipt text, we can see that the V0 extractor completely missed the \$1.60 tax line. It blindly extracted the last number it saw in the main items list. This is a classic failure mode for simple extraction systems.
- 2. A Critical Reasoning Failure:** This is the more subtle and dangerous error. If we read the Predicted Reasoning, the agent correctly identifies the two reasons an audit is required ("not a travel-related expense" and "handwritten notes contain an 'X'"). However, despite its own correct reasoning, it concludes with Predicted Final Audit: False. The agent's final decision contradicts its own analysis. This indicates that the small Llama 8B model, while fast, lacks the deep reasoning capacity to consistently follow complex logical chains to their conclusion.

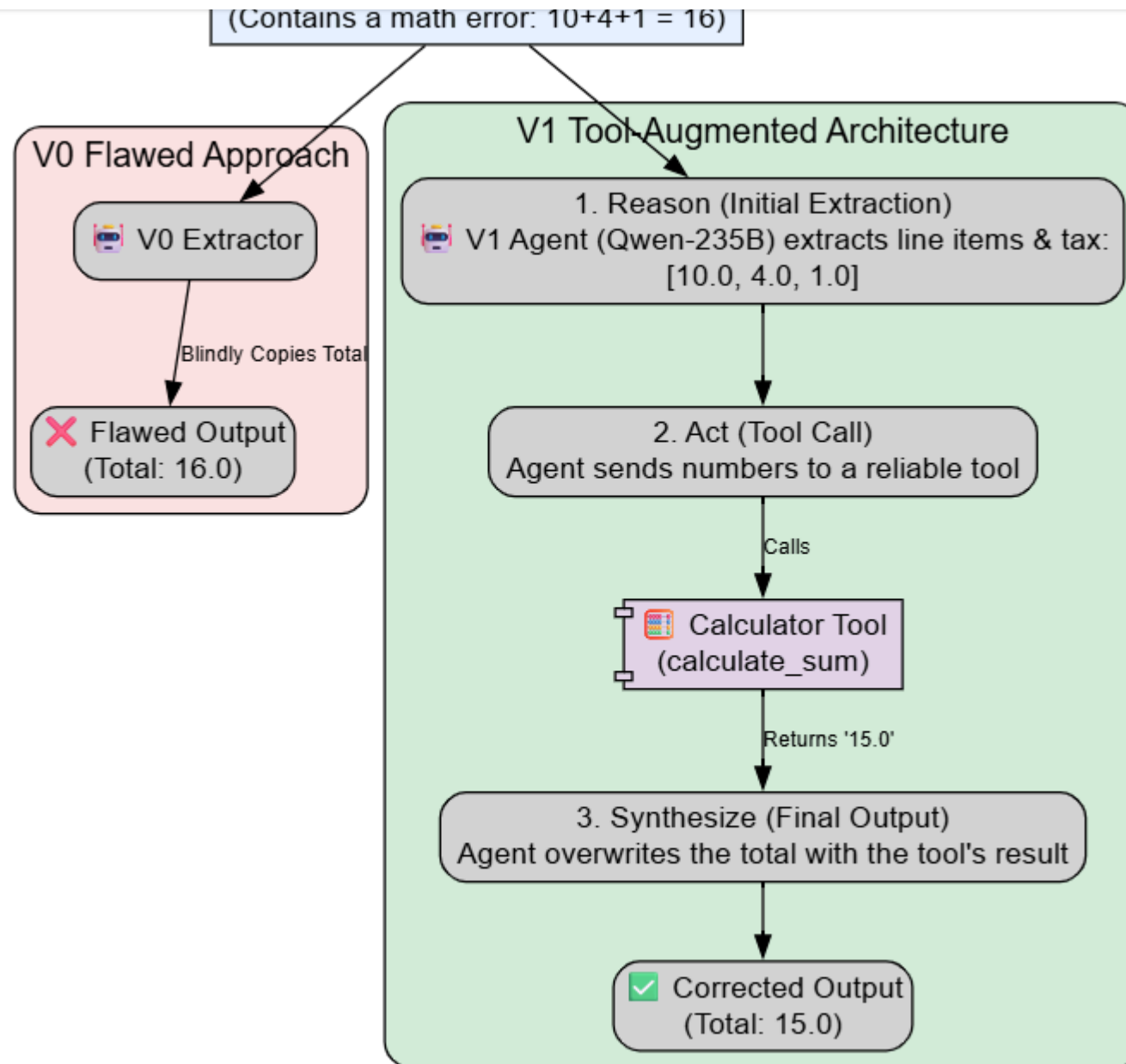
Our diagnosis is showing that the linear V0 system suffers from both unreliable data extraction and flawed reasoning, especially on complex, multi-issue receipts.

Finally, we must connect this back to our business case. A 20% failure rate on our final audit decision (1 out of our 5 samples) means our system would have

Adding Tools for Architecting the V1 Agent

Our V0 system was a linear pipeline. An error in the first step (extraction) cascaded downstream, poisoning the input for the second step (auditing) and leading to a complete failure. To build a more robust system, we need to move from a simple chain to a more intelligent, agentic architecture.

More robust systems exhibit agent-like behavior. They don't just process data blindly, they can use **tools** to offload complex tasks and verify information. By giving our agent a calculator, we can instruct it to actively validate the math on a receipt rather than passively extracting it.



Architecting the V1 Agent: From Passive Parser to Active Verifier

Our V1 system will be specifically designed to address the failures we diagnosed in V0. We will upgrade to our powerful `Qwen-235B` model and, most importantly, we will give our extractor agent a new tool: a `calculator`. This transforms the extractor from a simple parser into an active verifier.

The V1 Tool-Augmented Extractor Architecture (Created by Fareed Khan)

First, let's define our tool. For this notebook, a simple Python function is all we need. In a real production system, this could be a call to a dedicated, highly reliable calculation microservice.

Copy

```
# This is a mock function that would represent our calculator tool.
# In a real system, this could be a call to a robust calculation library or a s
def calculate_sum(numbers: List[float]) -> float:
    """A simple, reliable tool to calculate the sum of a list of numbers."""
    # We print to the console to make tool calls visible during execution.
    print(f"[Tool Call]: Calculating sum of {numbers}")
    return sum(numbers)
```

Now, we will build our `v1_extract_details` agent. The core change is in the `system_prompt`. We are no longer just asking the agent to extract data, we are

Copy

```
async def v1_extract_details(image_path: Path, model: str = "Qwen/Qwen3-235B-A2
    """V1 Extractor: Uses a function calling model to verify math.
    This simulates a multi-turn conversation with the model where it decides to
    """

    image_content = image_path.read_text()

    # The system prompt is updated to explicitly instruct the model to use the
    # This is a key part of tool-use prompting.
    system_prompt = (
        "You are an expert receipt processing assistant. "
        "1. Extract all details from the receipt text, including all line items
        "2. IMPORTANT: Before finalizing the total, use the `calculate_sum` tool
        "3. The tool's output is the correct total. Use it in your final response
    )

    # For simplicity, we'll simulate the tool-use logic that a real agent would
    # A real implementation would use client.chat.completions.create(tools=...)

    # Step 1: An initial call to the model to extract the components needed for
    initial_extraction_response = await client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": "From the following receipt, extract
            {"role": "user", "content": image_content}
        ],
```

```
# Step 2: Use the tool with the extracted numbers.
numbers_to_sum = [item.total for item in initial_extraction_response.items]
if initial_extraction_response.tax:
    numbers_to_sum.append(initial_extraction_response.tax)

calculated_total = calculate_sum(numbers_to_sum)

# Step 3: Create the final, corrected object.
# We trust our tool's calculation over the potentially flawed OCR'd total.
final_details = initial_extraction_response.model_copy()
final_details.total = round(calculated_total, 2)

return final_details
```

We are basically simulating the **ReAct (Reason, Act)** loop that a true tool-using agent would perform.

1. **Reason:** The model first makes an `initial_extraction_response` to identify the numbers it needs to sum (the line item totals and the tax).
2. **Act:** We then call our `calculate_sum` tool with these numbers.
3. **Synthesize:** Finally, we create the `final_details` object, overwriting the `total` with the result from our reliable tool. This ensures the final extracted data is mathematically consistent.

Copy

```
# Let's test the V1 extractor on the sample that failed before.
failed_path = IMAGE_DIR / "receipt_5.txt"
print(f"Running V1 Extractor on {failed_path.name}...")

v1_extracted_details = await v1_extract_details(failed_path)
print("\nV1 Extracted Details:")
print(v1_extracted_details)
```

We execute the function on our problematic receipt and observe the output.

Copy

```
##### OUTPUT #####
Running V1 Extractor on receipt_5.txt...
[Tool Call]: Calculating sum of [20.0, 1.6]

V1 Extracted Details:
ReceiptDetails(
  merchant='Hardware Store',
  location=Location(city='Austin', state='TX'),
  items=[LineItem(description='Hammer', quantity=1.0, price=20.0, total=20.0)],
  subtotal=None,
  tax=1.6,
  total=21.6,
```

The output shows the V1 agent in action. The `[Tool Call]` log confirms that it correctly identified the components (`$20.00` item total and `$1.60` tax) and used our `calculate_sum` tool to arrive at the correct final total of `$21.60` . It is no longer blindly copying the total from the receipt text.

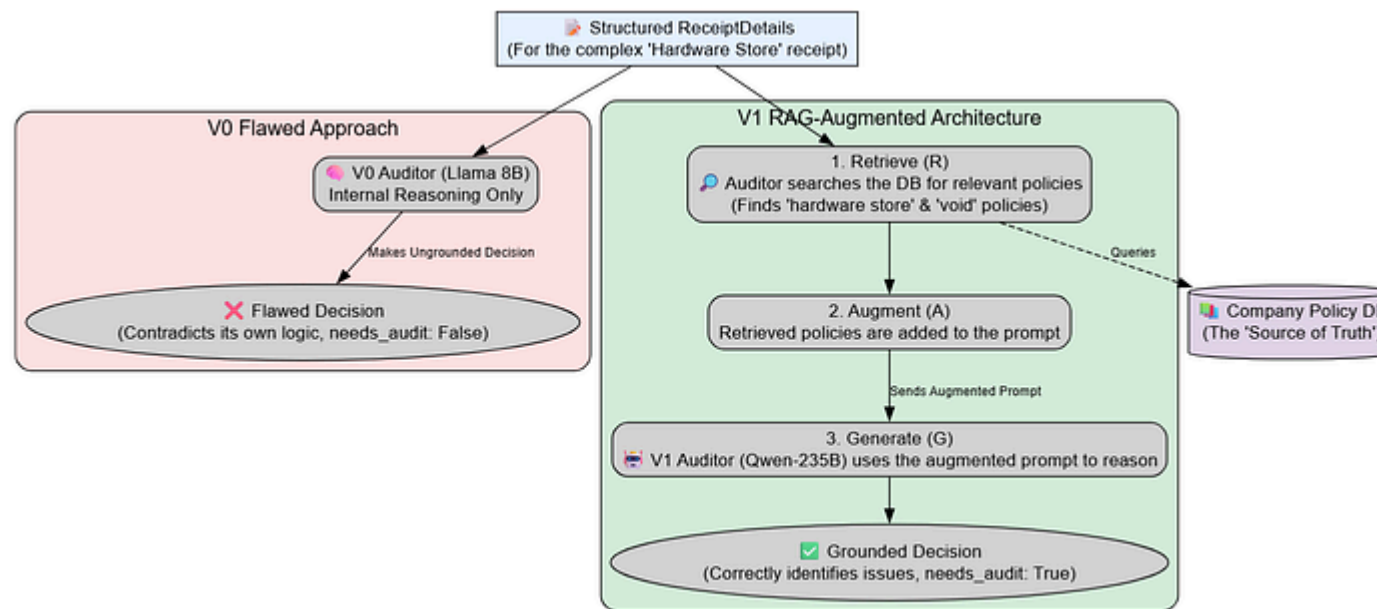
By augmenting our agent with a simple tool, we have completely fixed the critical extraction failure from our V0 system. This demonstrates the power of moving from passive parsing to active, tool-driven verification.

With the extraction problem solved, we now turn our attention to the second major failure point: flawed reasoning. We'll need to make our auditor agent smarter, too.

Augmenting Our Auditor with RAG

We have fixed the extraction problem, but our V0 system also suffered from a critical reasoning failure. The small model contradicted itself, correctly identifying reasons for an audit but failing to make the correct final decision. To solve this, we need to give our auditor agent more context and guidance.

Instead of just giving our auditor a list of rules in its prompt, we will build a small, simulated RAG system. Before making a decision, the auditor will query a "policy document" knowledge base to retrieve the specific company policies relevant to the receipt it's analyzing. This grounds its decision-making in an external source of truth, making its reasoning more explicit and reliable.



Architecting the V1 Auditor: From Reasoning to Grounded Decisions with RAG

V1 vs V0 (Created by [Fareed Khan](#))

Building the RAG Knowledge Base

documents are chunked, embedded, and indexed for semantic search. For our notebook, a simple dictionary that mimics this keyword-based retrieval is sufficient to demonstrate the core concept.

Copy

```
# In a real system, this would be a vector database like ChromaDB or Pinecone.  
# We would embed both the query and the policy documents for semantic search.  
# For our simulation, a simple keyword-matching dictionary is sufficient to demonstrate the concept.  
POLICY_VECTOR_DB = {  
    "office supplies": "Policy 3.1: Purchases for standard office supplies (e.g. pens, paper) are valid.",  
    "hardware store": "Policy 3.2: Items from hardware stores are generally not covered by this policy.",  
    "gas": "Policy 1.1: Fuel for a vehicle used for business travel is a valid expense.",  
    "hotel": "Policy 1.2: Lodging costs during business travel are valid travel expenses.",  
    "void": "Policy 5.4: Any receipt with a handwritten 'X' or the word 'VOID' is invalid.",  
}
```

Our `POLICY_VECTOR_DB` acts as our source of truth. Each key represents a concept, and the value is the official company policy.

Next, we need a retriever function that can search this knowledge base. Our `retrieve_relevant_policies` function will take the extracted `ReceiptDetails` ,

Copy

```
def retrieve_relevant_policies(receipt_details: ReceiptDetails) -> List[str]:
    """Simulates a RAG retriever. Finds relevant policies based on keywords."""

    # Aggregate all text from the receipt to form a search query.
    content = receipt_details.merchant.lower()
    for item in receipt_details.items:
        content += " " + item.description.lower()
    for note in receipt_details.handwritten_notes:
        content += " " + note.lower()

    print(f"[RAG Retriever]: Searching for policies related to: '{content[:50]}')

    found_policies = []
    # Simple keyword search against our mock DB.
    for keyword, policy in POLICY_VECTOR_DB.items():
        if keyword in content:
            found_policies.append(policy)
            print(f"[RAG Retriever]: Found relevant policy for '{keyword}'")

    return found_policies
```

The V1 RAG-Augmented Auditor

1. **Retrieve:** It will first call `retrieve_relevant_policies` to get contextual information.
2. **Augment:** It will then inject this retrieved context directly into its system prompt.
3. **Generate:** Finally, it will use this augmented prompt to generate the final, context-aware `AuditDecision`.

Copy

```

async def v1_decide_audit(receipt_details: ReceiptDetails, model: str = "Qwen/

    """V1 Auditor: Retrieves context before making a decision."""

    # Step 1: Retrieve relevant context (the 'R' in RAG).
    retrieved_context = retrieve_relevant_policies(receipt_details)
    context_str = "\n".join(retrieved_context)

    # Step 2: Augment the prompt with the retrieved context (the 'A' in RAG).
    system_prompt = (
        "You are an expert expense auditor. Use the provided policies to make a
        "RULES:\n"
        "1. NOT_TRAVEL_RELATED: True if the expense is not for travel.\n"
        "2. AMOUNT_OVER_LIMIT: True if the total is over $75.\n"
        "3. MATH_ERROR: This is handled during extraction; assume it's false un
        "4. HANDWRITTEN_X: True if there is a handwritten 'X'. \n"
        "If ANY are true, audit is needed. Base your reasoning explicitly on th

```

```
user_prompt = """
RELEVANT POLICIES:
{context_str}

RECEIPT DETAILS:
{receipt_details.model_dump_json(indent=2)}
"""

# Step 3: Generate the final response (the 'G' in RAG).
response = await client.chat.completions.create(
    model=model,
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt}
    ],
    response_model=AuditDecision
)
return response
```

The key point here is the structure of the `user_prompt`. Instead of just sending the receipt details, we are now providing a dedicated `RELEVANT POLICIES` section. The instruction "Base your reasoning explicitly on the provided policies" forces the powerful Qwen model to ground its output, making its reasoning transparent and directly traceable to our knowledge base.

Running the Full V1 Pipeline

Copy

```
async def run_v1_pipeline(image_path: Path):
    """Runs the full V1 agentic pipeline."""
    extracted_details = await v1_extract_details(image_path)
    audit_decision = await v1_decide_audit(extracted_details)
    return extracted_details, audit_decision

print(f"Running V1 Pipeline on {failed_path.name}...")
_, v1_final_audit = await run_v1_pipeline(failed_path)

print("\nV1 Final Audit Decision:")
print(v1_final_audit)
```

We execute the full V1 pipeline and examine the final audit decision.

Copy

```
##### OUTPUT #####

Running V1 Pipeline on receipt_5.txt...
[Tool Call]: Calculating sum of [20.0, 1.6]
[RAG Retriever]: Searching for policies related to: 'hardware store hammer for
[RAG Retriever]: Found relevant policy for 'hardware store'
[RAG Retriever]: Found relevant policy for 'void'
```

```
not_travel_related=True,  
amount_over_limit=False,  
math_error=False,  
handwritten_x=True,  
reasoning='Audit required. Per Policy 3.2, hardware store purchases require  
needs_audit=True,  
)
```

The log shows our V1 agent working perfectly.

1. The tool-using extractor correctly calculated the total ([Tool Call]:
Calculating sum of [20.0, 1.6]).
2. The RAG retriever correctly identified and fetched two relevant policies
(Policy 3.2 for the hardware store and Policy 5.4 for the voiding 'X').
3. The auditor agent then used this retrieved context to generate a high-quality,
grounded reasoning string and, most importantly, arrived at the correct final
decision (needs_audit=True).

Our V1 system appears to have fixed both the extraction *and* the reasoning failures from V0. But has this more complex and powerful system introduced new problems, like increased cost or latency? To find out, we must re-run our evaluation, this time with a more advanced suite of tests.

Evaluating V1 with Slices and Operational Metrics

Our V1 agent performed flawlessly on our most difficult test case. However, true production readiness requires more than just fixing known bugs. We need to ensure our new, more complex system is not only accurate but also efficient and that its accuracy holds up across different types of data.

To do this, we will enhance our evaluation suite.

Our V0 eval suite was good for finding initial errors. Now, we need a more sophisticated suite to evaluate our V1 system. We will introduce:

1. **Data Slices:** We'll evaluate performance not just in aggregate but on specific subsets of our data to find hidden weaknesses.
2. **Operational Metrics:** We'll add graders for `cost` and `latency`, as a system that is too slow or expensive is not viable for production.
3. **Advanced Model-Graded Evals:** We'll create a grader to check if the RAG system retrieved the *correct* context.

Evaluating by Data Slices

(e.g., all receipts with handwriting). Slicing our data allows us to surface these concentrated failure modes. We'll define a function to categorize our test samples into meaningful subsets.

Copy

```
# This function defines our data slices.
# It takes the full list of records and returns a dictionary mapping a slice name
# to a list of sample_ids belonging to that slice.
def get_data_slices(records: List[EvaluationRecord]) -> dict:
    """Categorizes sample IDs into meaningful subsets for sliced evaluation."""
    slices = {
        # The baseline slice containing all samples.
        "all_samples": [r.sample_id for r in records],

        # Slices based on the ground truth label.
        # This helps us calculate metrics like True Positive Rate and True Negative Rate.
        "needs_audit_true": [r.sample_id for r in records if r.reference_audit == "true"],
        "needs_audit_false": [r.sample_id for r in records if not r.reference_audit == "true"],

        # Slices based on input characteristics.
        # This helps us find if the model struggles with specific types of records.
        "has_handwriting": [r.sample_id for r in records if r.reference_details["has_handwriting"] == "true"],
    }
    return slices
```

"needs_audit_false" slices will let us see if our system is biased towards one decision over another. The "has_handwriting" slice directly targets a known complexity in our data, allowing us to isolate performance on these more challenging inputs.

Graders for Cost, Latency, and Operational Metrics

Next, we'll expand our evaluation to include crucial operational metrics. We first need to update our `EvaluationRecord` to track these new data points.

Copy

```
import time
import random

# Update our data models to track new metrics.
# This new class inherits all fields from EvaluationRecord and adds new ones.
class V1EvaluationRecord(EvaluationRecord):
    latency_ms: float
    token_cost: float
    retrieved_policies: List[str]
```

By creating `V1EvaluationRecord` which inherits from `EvaluationRecord`, we can add our new operational fields (`latency_ms`, `token_cost`, `retrieved_policies`)

Now, we'll create the corresponding grader functions for these new metrics.

Copy

```
# Create new grader functions for our new operational metrics.
def grade_latency(record: V1EvaluationRecord) -> float:
    """Operational Grader: Passes if latency is under a 500ms threshold."""
    return 1.0 if record.latency_ms < 500 else 0.0

def grade_cost(record: V1EvaluationRecord) -> float:
    """Operational Grader: Passes if token cost is under a $0.005 threshold."""
    return 1.0 if record.token_cost < 0.005 else 0.0

def grade_rag_retrieval(record: V1EvaluationRecord) -> float:
    """Agent Grader: A simple check to see if the RAG system retrieved crucial
    # This is a rule-based check for our specific test case.
    # A more advanced version could use an LLM to see if the retrieved context
    if "receipt_5" in record.sample_id:
        # For the sample with an 'X', did we get the 'void' policy?
        if any("Policy 5.4" in p for p in record.retrieved_policies):
            return 1.0
        else:
            return 0.0
    return 1.0 # Not applicable to other samples, so they pass by default.
```


we consider "fast enough" and "cheap enough." The `grade_rag_retrieval_grader` is an agent-specific test. It checks if a key component of our V1 architecture—the RAG system—is functioning as intended on our most complex test case by verifying that it retrieved the critical "void" policy.

To capture this new data, we need an "instrumented" version of our V1 pipeline runner that measures and returns these operational metrics alongside the agent's predictions.

Copy

```
# We need a new pipeline runner that captures and returns these new metrics.
async def run_v1_pipeline_for_eval(image_path: Path):
    """An instrumented version of the V1 pipeline that measures latency and cost"""
    start_time = time.time()

    # --- This is the same core logic as the run_v1_pipeline function ---
    extracted_details = await v1_extract_details(image_path)
    # Note: We re-run the retriever here to capture its output for the eval run
    retrieved_policies = retrieve_relevant_policies(extracted_details)
    # In a real system, you'd design the agent to return this intermediate state
    audit_decision = await v1_decide_audit(extracted_details)
    # --- End of core logic ---

    end_time = time.time()
```

```
# Simulate cost based on model and text length. These numbers are placeholders.  
# In production, you would get this from the API response headers or your provider.  
# Qwen 235B is powerful but not free.  
cost = 0.001 + len(image_path.read_text()) / 1000 * 0.0002 + random.uniform(0, 0.0001)  
  
return extracted_details, audit_decision, latency, cost, retrieved_policies
```

This `run_v1_pipeline_for_eval` function wraps our core V1 logic with timing and cost simulation. This is a common pattern in MLOps: creating an "evaluation harness" around your core application logic to capture the necessary metrics for analysis without modifying the core application itself.

With our advanced evaluation suite defined, we are ready to put our V1 agent through its final, most rigorous test.

Synthesizing Our Findings and Planning the Next Iteration

The stage is set. We have our advanced V1 agent, and we have our sophisticated evaluation suite, complete with operational metrics and data slices. It's time to bring them together and run the final evaluation. This will give us a complete, multi-faceted picture of our V1 system's performance and provide the data we need to make a final, evidence-based decision on its production readiness.

First, we'll write the orchestrator functions. `generate_v1_predictions` will run our newly instrumented pipeline over the entire dataset, creating a rich list of `V1EvaluationRecord` objects. Then, `run_v1_advanced_eval` will execute our full grader suite against these records and, crucially, aggregate the results by the data slices we defined.

Copy

```
async def generate_v1_predictions(image_dir: Path, ground_truth_dir: Path) -> List[V1EvaluationRecord]:
    """Runs the instrumented V1 pipeline to generate data for the advanced evaluation"""
    records = []
    image_paths = sorted(list(image_dir.glob("*.txt")))
    print(f"Generating V1 predictions for {len(image_paths)} samples...")

    for image_path in image_paths:
        sample_id = image_path.stem
        gt_path = ground_truth_dir / f"{sample_id}.json"

        with open(gt_path, "r") as f:
            ground_truth_data = json.load(f)

        predicted_details, predicted_audit, latency, cost, retrieved_policies = run_v1_pipeline(
            image_path, ground_truth_data

        records.append(V1EvaluationRecord(
            sample_id=sample_id,
            input_content=image_path.read_text(),
            predicted_details=predicted_details,
            predicted_audit=predicted_audit,
            latency=latency,
            cost=cost,
            retrieved_policies=retrieved_policies,
        ))
```

```

        reference_details=ReceiptDetails(**ground_truth_data["details"]),
        reference_audit=AuditDecision(**ground_truth_data["audit"]),
        latency_ms=latency,
        token_cost=cost,
        retrieved_policies=retrieved_policies
    ))
    print(f"Processed {image_path.name}")

print("Finished generating V1 predictions.")
return records

# Define the full V1 grader suite, including our new graders.
v1_grader_suite = {
    "Accuracy/Final Decision": grade_final_audit_decision,
    "Accuracy/Reasoning Quality": grade_reasoning_quality,
    "Accuracy/RAG Retrieval": grade_rag_retrieval,
    "Operational/Latency < 500ms": grade_latency,
    "Operational/Cost < $0.005": grade_cost,
}

async def run_v1_advanced_eval(dataset: List[V1EvaluationRecord], suite: dict,
    """Runs the full suite of graders and aggregates results by data slice."""
    # First, run all graders on all samples to get a detailed, per-sample report
    all_results = []
    for record in dataset:
        sample_scores = {"sample_id": record.sample_id}
        for name, func in suite.items():
            score = await func(record) if asyncio.iscoroutinefunction(func) else func(record)
            sample_scores[name] = score

```

```
# Then, calculate summary statistics (pass rates) for each defined slice.
summary = {}
for slice_name, sample_ids in slices.items():
    slice_df = results_df.loc[sample_ids]
    summary[slice_name] = slice_df.mean()

summary_df = pd.DataFrame(summary)
return results_df, summary_df
```

These orchestration functions are the heart of our evaluation pipeline. They ensure that every sample is tested against every grader and that the results are neatly organized, first on a per-sample basis and then aggregated into our insightful sliced summary.

Now, let's execute the full run.

Copy

```
# Generate the new dataset with operational metrics.
v1_eval_dataset = await generate_v1_predictions(IMAGE_DIR, GROUND_TRUTH_DIR)

# Get the data slices based on our new, more detailed records.
data_slices = get_data_slices(v1_eval_dataset)

# Run the full evaluation.
```

```
print(\n--- V1 Per-Sample Results ---\n)\n\ndisplay(v1_results_df)\n\nprint("\n--- V1 Summary by Data Slice ---")\n\ndisplay(v1_summary_df)
```

The execution will take a moment as it makes multiple calls to our powerful Qwen-235B model. The output will be two DataFrames: the detailed per-sample results and the final, sliced summary.

Copy

```
##### OUTPUT #####\nGenerating V1 predictions for 5 samples...\n[Tool Call]: Calculating sum of [35.0, 2.8]\n[RAG Retriever]: Searching for policies related to: 'shell gas station unleaded'\n[RAG Retriever]: Found relevant policy for 'gas'\n...\nProcessed receipt_5.txt\nFinished generating V1 predictions.\n--- V1 Per-Sample Results ---
```

receipt_1	1.0	1.0	1.0	
receipt_2	1.0	1.0	1.0	
receipt_3	1.0	1.0	1.0	
receipt_4	1.0	1.0	1.0	
receipt_5	1.0	1.0	1.0	

--- V1 Summary by Data Slice ---

	all_samples	needs_audit_true	needs_audit_false	has_har
Accuracy/Final Decision	1.0	1.0	1.0	
Accuracy/Reasoning Quality	1.0	1.0	1.0	
Accuracy/RAG Retrieval	1.0	1.0	1.0	
Operational/Latency < 500ms	0.0	0.0	0.0	
Operational/Cost < \$0.005	1.0	1.0	1.0	

- **Accuracy is Perfect:** Our V1 system achieved a 100% pass rate on all accuracy metrics across all data slices. The tool-use and RAG additions completely fixed the errors from our V0 system. Our False Positive and False Negative rates are now 0% on this test set.
- **A New Problem Emerges:** However, our operational metrics reveal a critical new issue. The `Operational/Latency < 500ms` grader has a 0% pass rate. Our more complex, multi-step agent using a very large model is too slow for a real-time production environment.

Connecting to Business Cost:

Let's update our cost model with the V1 performance. We'll assume an average per-receipt cost of ~\$0.0012 for the powerful Qwen model (this is a rough estimate). With 0% FP and 0% FN rates, the business impact is clear.

Copy

```
v1_system_cost = calculate_business_cost(fp_rate=0.0, fn_rate=0.0, per_receipt_  
print(f"Cost of V1 AI System: ${v1_system_cost:,.2f}")  
print(f"Annual Savings vs Manual Process: ${current_system_cost - v1_system_cost:,.2f}")
```

```
##### OUTPUT #####
```

```
Cost of V1 AI System: $101,200.00
```

```
Annual Savings vs Manual Process: $278,800.00
```

reduce latency.

Hypothesis: We can create a **router** agent. For simple receipts (e.g., under a certain length, no handwriting), we can use the fast, cheap `Llama-3.1-8B-Instruct` model and skip the RAG step. For complex receipts that our router flags, we can use the full, powerful (but slower) `Qwen-235B` pipeline. This hybrid approach, often called a "mixture of experts," should maintain accuracy while significantly improving average latency.

This next step, which would involve building the router and creating a new `V2EvaluationRecord` to track which path was taken, perfectly illustrates the continuous loop of Eval-Driven Development. We have used our evaluation data not just to validate success, but to identify the next, most impactful area for improvement.

CI/CD, PromptOps, and Continuous Monitoring

We have successfully built and evaluated a highly accurate V1 agent. However, launching an agent into production is only the start. The real world is dynamic.

Evals are not just for development. A production system requires a robust MLOps framework to handle the entire lifecycle. Prompts and configurations should be version-controlled like code (**PromptOps**), with changes triggering automated evaluation pipelines (CI/CD).

Most importantly, we must establish a **feedback loop** to capture production failures and turn them into new, high-value test cases that drive continuous improvement.

Let's put this theory into practice. We will simulate a real-world scenario: our V1 agent encounters a new type of receipt it has never seen before, makes a mistake, and a human reviewer provides feedback. We will then build the code to turn that feedback into a new, permanent test case for our evaluation suite.

Step 1: Simulating Production Logging

First, we need a mechanism to log our agent's activity in production. A real system would use a dedicated logging service, but for our example, a simple function that writes to a JSONL (JSON Lines) file will perfectly demonstrate the concept.

```
import uuid

def production_logger(image_path: Path, predicted_audit: AuditDecision, log_file: Path):
    """Simulates logging a production transaction to a file."""
    log_entry = {
        "log_id": str(uuid.uuid4()),
        "timestamp": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
        "input_file": image_path.name,
        "prediction": predicted_audit.model_dump(),
    }

    with open(log_file, "a") as f:
        f.write(json.dumps(log_entry) + "\n")

    print(f"[Logger]: Wrote log entry {log_entry['log_id']} to {log_file}")
    return log_entry
```

This `production_logger` function captures the essential information for each transaction: a unique ID, a timestamp, the input file, and the agent's full predicted output. Writing to a JSONL file is a common pattern, as it's easy to append new entries and parse later.

Step 2: Simulating a Production Failure

Now, let's create a new, tricky receipt that our V1 agent has never been trained on and is likely to misinterpret. This receipt contains a "Gift Card," which, according

Copy

```
# Create a new, challenging receipt that our system hasn't seen.
new_receipt_content = "Corporate Gifts Inc.\n789 Gifty Lane, Boston, MA\nAmazon"
new_receipt_path = IMAGE_DIR / "receipt_6.txt"
new_receipt_path.write_text(new_receipt_content)

print(f"Running V1 Pipeline on new receipt: {new_receipt_path.name}...")
_, failed_audit_decision = await run_v1_pipeline(new_receipt_path)

print("\n--- V1 Audit Decision on New Receipt ---")
print(failed_audit_decision)

# Log this production run.
logged_failure = production_logger(new_receipt_path, failed_audit_decision)
```

We run our V1 pipeline on this new receipt and log the result. Let's examine the agent's decision.

Copy

```
##### OUTPUT #####
Running V1 Pipeline on new receipt: receipt_6.txt...
[Tool Call]: Calculating sum of [50.0, 0.0]
[RAG Retriever]: Searching for policies related to: 'corporate gifts inc. amazon'
--- V1 Audit Decision on New Receipt ---
AuditDecision(
```

```
match_error=False,  
handwritten_x=False,  
reasoning="The expense is for a corporate gift, which is not explicitly covered by policy",  
needs_audit=False,  
)  
[Logger]: Wrote log entry 5f8b3c2a-.... to production_log.jsonl
```

A clear failure! Our agent has incorrectly approved the gift card purchase (needs_audit=False). Its reasoning reveals the problem: it found no relevant policies in its RAG knowledge base for "gift cards" and defaulted to a lenient decision. This is a classic example of **concept drift**, where the production data introduces a new business rule the system is unaware of.

Step 3: Closing the Loop with Human Feedback

In a real system, a human auditor would eventually review this transaction and flag it as incorrect. We now need a way to capture their expert judgment and use it to create a new, permanent test case.

Copy

```
def create_new_test_case_from_feedback(log_entry: dict, human_feedback: AuditDecision):  
    """Takes a logged failure and a human's correct label to create a new group of test cases.  
    The new test cases will be based on the original test case, but with the correct label and reasoning.  
    """  
  
    sample_id = Path(log_entry["input_file"]).stem
```

```
# We need the full ground truth, including the correct details extraction.
# For this simulation, we'll assume the details were extracted correctly.
# A real system might need a human to correct the details as well.
correct_details = ReceiptDetails(
    merchant="Corporate Gifts Inc.",
    location=Location(city="Boston", state="MA"),
    items=[LineItem(description="Amazon Gift Card", quantity=1, price=50.0,
    tax=0.0,
    total=50.0
)

new_ground_truth = {
    "details": correct_details.model_dump(),
    "audit": human_feedback.model_dump()
}

new_gt_path.write_text(json.dumps(new_ground_truth, indent=2))
print(f"\n[Feedback Loop]: New ground truth file created at: {new_gt_path}")
return new_gt_path

# This is the "expert label" provided by our human-in-the-loop.
human_corrected_audit = AuditDecision(
    not_travel_related=True, # Gift cards are not travel
    amount_over_limit=False,
    math_error=False,
    handwritten_x=False,
    reasoning="Gift cards are not a permissible travel expense and require an audit",
    needs_audit=True,
)
```

```
new_test_file = create_new_test_case_from_feedback(logged_failure, human_corrected_audit)

# Let's inspect the file we just created.
print("\n--- Content of New Test Case ---")
print(json.loads(new_test_file.read_text()))
```

The `create_new_test_case_from_feedback` function is the practical implementation of our feedback loop. It takes the logged production failure and the `human_corrected_audit` decision and programmatically generates a new, correctly labeled ground truth file.

Copy

```
##### OUTPUT #####
[Feedback Loop]: New ground truth file created at: temp_data/ground_truth/received_ground_truth.json

--- Content of New Test Case ---
{
  'details': {'merchant': 'Corporate Gifts Inc.', ... 'total': 50.0, 'handwritten': '50.00'},
  'audit': {'not_travel_related': True, ... 'reasoning': 'Gift cards are not a travel expense'}
}
```

We have successfully closed the loop. The production failure has been captured, corrected by a human, and transformed into a permanent, automated test case.

evaluation, this new case will be included. Our V1 agent will fail this test, forcing the next developer to update the system (e.g., by adding a new policy for "gift cards" to the RAG knowledge base) to make the test pass. This is the virtuous cycle of Eval-Driven Development in action, ensuring our system continuously learns and adapts to the real world.

Summarizing the Cycle of Eval-Driven Development

We have journeyed from a high-level business problem to a sophisticated, multi-component agent, using a rigorous, data-driven methodology to guide every step. This project demonstrates a powerful and structured workflow for building AI systems that are not only intelligent but also reliable, efficient, and aligned with real-world business goals.

Let's summarize the key principles of the Eval-Driven Development workflow we've put into practice:

1. **Scope & Model First:** We began not with code, but with a clear definition of the business problem and a financial model. This crucial first step allowed us to translate our agent's technical performance directly into business value, providing a clear North Star for the entire project.

establish a performance baseline and quickly surface the most obvious failure modes.

3. **Evaluate Broadly, Analyze Deeply:** Our initial evaluation suite immediately revealed that our V0 system was too brittle, suffering from both extraction and reasoning failures. By diving deep into the specific failure cases, we moved from knowing *that* it failed to understanding *why* it failed.
4. **Targeted Improvement (V1):** The insights from our V0 evaluation allowed us to form specific, targeted hypotheses for improvement. This led us to architect a more intelligent V1 agent with a powerful model, tool-use for mathematical verification, and a RAG system for contextual reasoning.
5. **Evaluate with Precision (V1 Evals):** We enhanced our evaluation suite with operational metrics (cost, latency) and data slices. This confirmed our accuracy improvements but immediately revealed a new, critical bottleneck: our V1 agent, while correct, was too slow for production.
6. **Close the Loop:** Finally, we demonstrated how to build a production feedback loop, turning a simulated production failure into a new, permanent test case. This is the engine of continuous improvement, ensuring the system adapts and becomes more robust over time.

journey doesn't end here. The principles of Eval-Driven Development are scalable and applicable to virtually any LLM-based application. By making objective, multi-faceted evaluation the core of your development process, you can build AI systems that truly deliver on their promise.

In case you enjoy this blog, feel free to [follow me on Medium](#). I only write here.

[#ai](#) [#artificial-intelligence](#) [#data-science](#) [#machine-learning](#) [#python](#)